random numbers? Create dungeons and maps on the fly, control the player's progression through these worlds, have enemies scaling up accordingly, and create a new experience every time they boot up a game? A single playthrough of such games could range between 30 minutes to maybe 3 hours. But the replayability value is infinite. Every experience is unique, and every decision was taken into account with what the game provides in that specific playthrough or run. It opens up a world of possibilities, and it can be done without handcrafting all this content for the audiences.

Surprisingly, this had already been explored before by enterprising game developers of the retro era leading the Indie developers to turn to old relics like Elite (1984) and Rogue (1980) which utilised random number generation (RNG) and procedural generation. A lot of space sims chalk up Elite as their inspiration including Eve Online (2003) and No Man's Sky (2016), while Rogue by itself was so conceptually practical and unique that catalysed an entire genre of games, decades later after its release.

Procedural Generation

Procedural generation is one of the most popular applications of RNG. Procedural generation (aka Procgen) is when a computer generates content in a semi-random manner rather than by hand by a human. Procedural generation techniques often create environments, monster encounters, and treasure drops. Procgen is a potent tool, and when used correctly on the right project, it may drastically improve a game's quality and lengthen its longevity by enhancing its replayability. The players aren't forced to replay the same material over and over. Procedural content can assist in keeping them engaged for much longer. This is the most obvious way that procgen can help you improve your game since it allows you to produce content for your users in near-infinite ways. Take a look at the durability of games like Spelunky, Dead Cells and others that make good use of procedural level generation. People have poured hundreds of hours into these games, and they have aged like fine wine, with their gameplay experience still unique, even with the advent of new titles.

Elite: A retro space sim

Procedural generation techniques aren't just restricted to level generation, and they have various use cases. Shadow of Mordor leverages procedural generation to generate enemy orcs. The notion allows gamers to form bonds with orcs they face in battle. Every orc is distinct from their names and looks to their speech patterns and interactions with other orcs, the game's Nemesis System has built them all algorithmically. Every encounter the player has with a specific orc leaves behind a permanent change that they can notice on the subsequent encounter giving the impression that they're fighting particular and unique archrivals rather than the identically programmed sub-bosses in many other games. Cameron Browne took procgen to another level by creating a program called Ludi that generated game rules procedurally. Ludi generates games by taking the rules of current games and scrambling them into new combinations using genetic programming (G.P.) techniques of crossover and mutation. New games are evaluated via self-play trials and given a quality score based on their anticipated ability to entice gamers; hence, the creation, testing, and assessment process are automated. Ludi uses a Markovian method seeded with Tolkien-style terms to develop a unique name for each evolving game. Yavalath was eventually published as a board game in 2009.


Terraria uses procgen to create natural terrain in 2D

The way procgen systems leverage randomness to create unlimited content is interesting. There are several problems that the developers have to tackle. As mentioned before, modern computers are good at following rigid instructions, and randomness as a concept is very foreign to the fundamentals of how computers work. Simulating randomness is computationally expensive and is still a mammoth task to this day. Hence, computers use pseudo-randomness to make it computationally cheap and practical to simulate randomness. Another challenge procgen systems pose is manipulating randomness into something that makes sense. If Minecraft developers just used random numbers, it would look something like the first image on the next page.

Creating something complicated and chaotic yet structured and